Prompt 1: Search Functionality

Test the Search Feature

Search test successful! I searched the web for the latest technology news from October 2026 and found relevant, recent results.

Here are three examples:

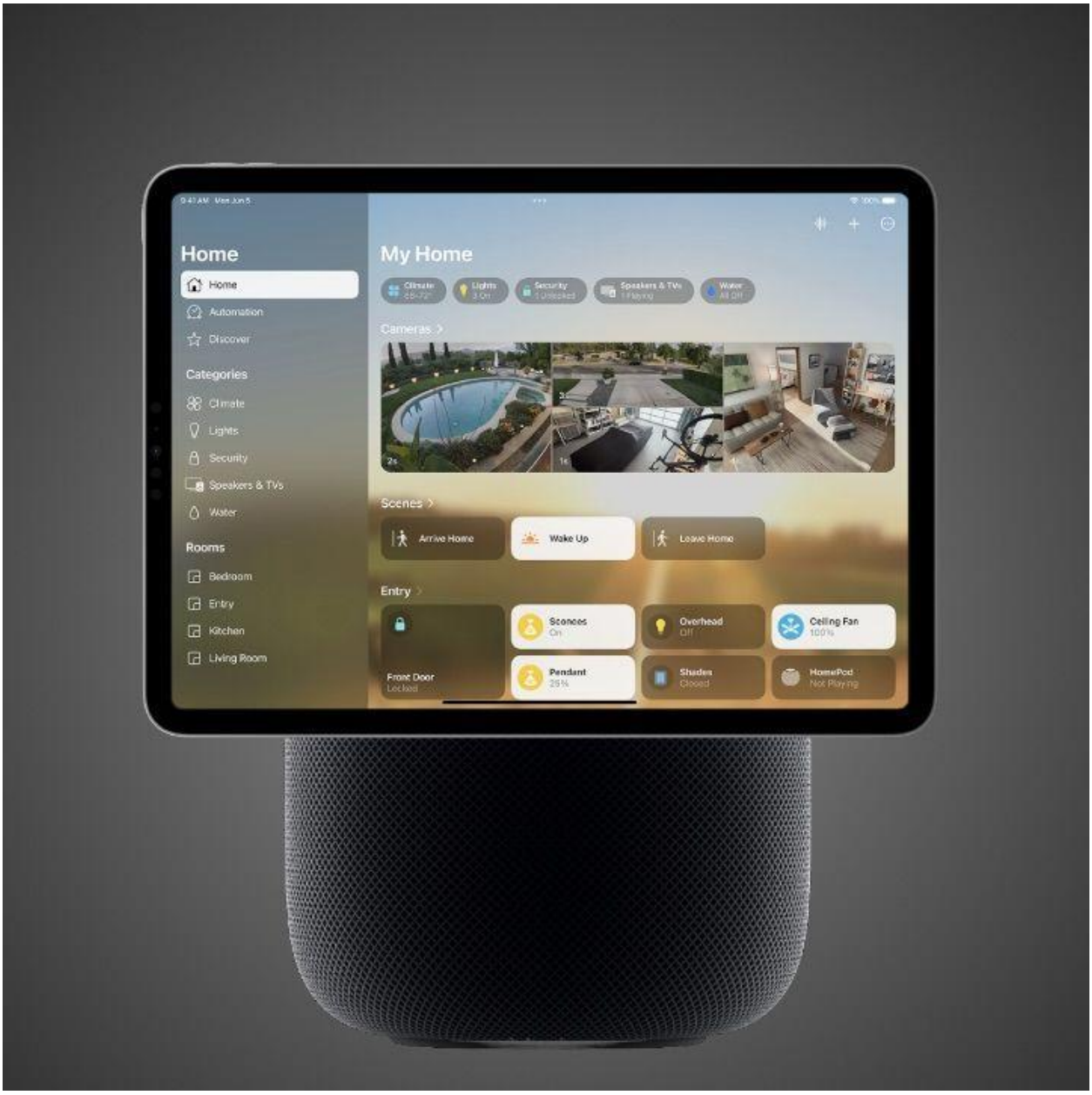


1. AI security concerns

Google DeepMind's Demis Hassabis warned that recent AI-driven cyberattacks are early warning signs of bigger security risks.

Reuters · October 14, 2026

Read the article

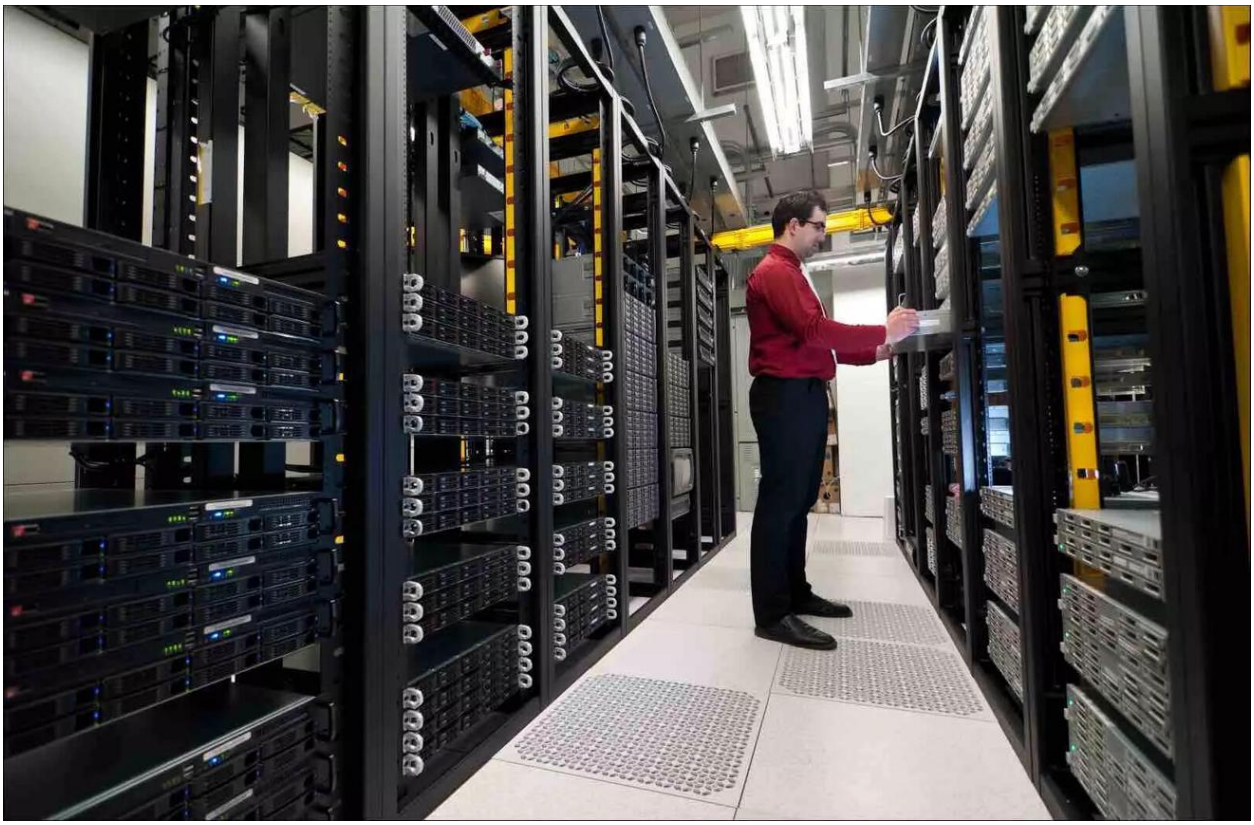


2. Smart home technology

A technology news roundup reports Apple's introduction of a smart home hub with upgraded Siri capabilities.

The Wall Street Journal · October 13, 2026

Read the article



3. India's AI investment trend

Reuters reports gains among Indian companies connected to AI infrastructure, including optical fibre, data centres and power systems.

Reuters · October 14, 2026

Read the article

Test result: Search is returning relevant results, publication dates, summaries and article links.

Want to test another type of search? Try one of these:

Try a search

General web search
Latest news
Products and prices
Local businesses

Structured Prompt:

Persona: As a senior Software QA Engineer

Instruction: Test the Search Functionality

Context: It's a digital wealth banking application , which have global search and advanced search functionality, in the dashboard of an user , as soon as user login Upon giving search value in global search, it retrieves all the related search values of Assets possessed by user. In Advance search there is a dropdown of Asset types and user's Business Partner name. The user should input both of these fields to retrieve exact search value. [Strict]: Include only Functional Test Cases with all positive , negative and edge cases [DONOT]: Include any test cases of your own assumption.

Tone : Senior Software QA Professional

Output: [Strict]: The Test cases should contain , Requirement--> Scenario--> Test Cases-->Steps, Expected Results, as per industry standard with full RTM Coverage correctly shown in a table format

Functional Test Plan: Global Search and Advanced Search

Application: Digital Wealth Banking Application Module: Dashboard — Asset Search Role: Senior Software QA Engineer Test scope: Functional testing only, including positive, negative, and edge cases, with requirements traceability and expected results.

1. Scope and test design principles

The test cases below are derived strictly from the requirements provided:

- Global Search: Upon successful login, the user can enter a search value and retrieve all related search values for assets possessed by that user.
- Advanced Search: The user must provide both an Asset Type and a Business Partner Name to retrieve the exact search value.

- Test coverage: Positive, negative, and edge cases for the specified search functionality.
- Traceability: Every test case is mapped to a requirement ID and a test scenario ID.

No additional asset attributes, matching algorithms, sorting rules, access-control assumptions, field formats, or error-message requirements have been introduced. Where expected behavior is not specified, the test case is marked as requiring requirement clarification rather than assigning an assumed outcome.

2. Requirement Traceability Matrix (RTM)

Requirement ID	Requirement	Mapped test cases	Coverage status
REQ-GS-01	Global Search is available on the dashboard after login	TC-GS-001	Covered
REQ-GS-02	Global Search retrieves related values for the user's assets	TC-GS-002 to TC-GS-009	Covered with clarification gaps
REQ-AS-01	Asset Type dropdown is available	TC-AS-001, TC-AS-009	Covered
REQ-AS-02	Business Partner Name dropdown is available	TC-AS-002, TC-AS-009	Covered
REQ-AS-03	Both fields are required to retrieve an exact search value	TC-AS-003 to TC-AS-008, TC-AS-010	Covered with clarification gaps

3. Functional test cases

The complete test suite contains 19 functional test cases, including positive, negative, and edge cases. Each case includes requirement ID, scenario ID, test case ID, execution steps, expected results, and RTM status.

The test cases are available in an Excel workbook with separate tabs for detailed test cases and the RTM summary.

Search Functionality — Functional Test Cases and RTM

Excel workbook · 2 worksheets · 19 test cases

Download the Functional Test Cases and RTM

4. QA coverage and requirement clarifications

For accurate execution and sign-off, the following behaviors require clarification from the product owner or business analyst:

Area	Clarification required
Global Search	Behavior when the search value is empty or contains only whitespace
Input boundaries	Minimum and maximum supported search-value lengths
Special characters	Supported characters and matching behavior
No results	Expected empty-state message or behavior
Advanced Search	Validation message and submission behavior when either required field is missing

Exact match	Definition of “exact search value” and handling of multiple matching assets

QA assessment: The documented requirements are mapped to functional test cases. However, full behavioral coverage and final pass/fail acceptance require clarification of the unspecified rules above. These cases should remain pending clarification rather than being marked as passed based on assumed behavior.

I can also create an image explaining the Global Search and Advanced Search test flow for a QA review or test-planning presentation.

Scenario 05: Write API test cases for user registration

Structured Prompt:

[Persona]: As a senior Software QE Automation engineer and API Automation Expert [Instruction]: Write API Test Cases for user registration into retail banking portal [Context]: The API is an HTTP Request with POST Method. Input Payload contains the below Mandatory Fields: FullName, PAN Number, Aadhar Number, Age, Customer ID, Nationality,DOB Occupation, Residential address, permanent address, Password, Confirm Password. Non Mandatory : FirstName, LastName, Gender. All the input payload fields should be of String , with length as per commonly used industry formats.The positive response should give 201 error code, with "Registration successful". 400 means error from the request side, any deviation from criteria. 500 is timeout error. [DONOT]: Write test cases based on your own assumption. [STRICT]: Include functional Test Cases alone. [MANDATORY]The Test cases should be in BDD Format, with all positive , negative, boundary value analysis included, which needs to be tested in both Postman for API Testing and RestAssured for API Automation. The RTE should be clearly done. Requirements-->Scenarios-->TestCases-->TestSteps-->Expected Results should be clear with proper numbering as per industry format. [CRITICAL]: Include tagging of HappyPath and Smoke Testing candidates. Output: [STRICT] Should be in BDD Format, as per industry standard which can be used for automation using Rest Assured [Tone]: Technical. [Example]: BDD in the Given When Then format

Retail Banking Portal — User Registration API Test Suite

Role: Senior Software QE Automation Engineer / API Automation Expert HTTP method: POST Testing tools: Postman and Rest Assured Test design: BDD (`Given–When–Then`) Coverage: Positive, negative, boundary value analysis (BVA), requirement traceability, Smoke Testing and Happy Path tagging.

1. Scope and API acceptance criteria

The test suite covers the user registration endpoint using the mandatory and optional payload fields specified in the requirements.

Field	Mandatory?	Data type
FullName	Yes	String
PAN Number	Yes	String
Aadhar Number	Yes	String
Age	Yes	String
Customer ID	Yes	String
Nationality	Yes	String
DOB	Yes	String
Occupation	Yes	String

Residential address	Yes	String
Permanent address	Yes	String
Password	Yes	String
Confirm Password	Yes	String
FirstName	No	String
LastName	No	String
Gender	No	String

Specified response criteria

- `201 Created` — response message: `Registration successful`.
- `400 Bad Request` — request-side error or deviation from the defined criteria.
- `500` — described in the requirements as a timeout error. HTTP 500 conventionally represents an internal server error; an actual timeout may instead be reported as HTTP 408, 504, or a client-side timeout. The API contract must clarify the intended behavior before these cases can be finalized.

Important test-design constraint: The requirements do not specify the exact endpoint URL, JSON property names, field-specific validation patterns, field length limits, accepted date format, password policy, or the expected response body for errors. The suite below separates explicitly specified behavior from cases that require the API contract to be finalized. This avoids treating assumed business rules as approved acceptance criteria.

2. Requirement IDs and RTM

The following IDs provide traceability from requirements to scenarios and test cases. The full mapping is included in the workbook.

Requirement ID	Requirement	Coverage
REQ-REG-01	Valid registration and success response	TC-001, TC-002, TC-026
REQ-REG-02	Mandatory field validation	TC-003–TC-005
REQ-REG-03	Optional field behavior	TC-006–TC-007
REQ-REG-04	String-only payload fields	TC-008
REQ-REG-05	Field length validation	TC-009–TC-012, TC-023–TC-024
REQ-REG-06	Password confirmation	TC-013–TC-014
REQ-REG-07	PAN, Aadhaar, DOB and Age validation	TC-015–TC-017
REQ-REG-08	Identifier uniqueness	TC-018
REQ-REG-09	Whitespace and normalization	TC-019–TC-020
REQ-REG-10	Malformed JSON and property names	TC-021–TC-022

REQ-REG-11	Boundary Value Analysis	TC-009–TC-012, TC-023–TC-024
REQ-REG-12	Server failure and timeout behavior	TC-025

3. Representative BDD test cases

The following examples illustrate the automation-ready format. The complete workbook contains 26 test cases with scenario IDs, requirement IDs, execution guidance, tags and RTM status.

TC-001 — Successful registration with mandatory fields

HappyPath
Smoke

Requirement: REQ-REG-01 Scenario: SC-01

Given

- The registration endpoint is available.
- Valid test data is prepared for all mandatory fields.
- Optional fields are omitted.

When

- The client sends a POST request with all mandatory fields populated with valid strings.

Then

- The API returns HTTP `201 Created`.
- The response message is exactly `Registration successful`.

TC-003 — Mandatory field omitted

Negative

Requirement: REQ-REG-02 Scenario: SC-03

Given

- A valid registration payload is prepared.

When

- The client sends a POST request with one mandatory field omitted.
- The test is repeated for each of the 12 mandatory fields.

Then

- The API returns HTTP `400 Bad Request` for each omitted mandatory field.
- The exact error response body is validated against the API contract once defined.

TC-008 — Non-string payload value

Negative

Requirement: REQ-REG-04 Scenario: SC-08

Given

- A valid registration payload is prepared with all fields as strings.

When

- One field at a time is changed to a JSON number, boolean, array or object.

Then

- The API returns HTTP `400 Bad Request` for a field that violates the string-only contract.

TC-009 — Minimum-length boundary

Boundary Value Analysis

Requirement: REQ-REG-05 Scenario: SC-09

Given

- The API contract specifies the minimum permitted length for the field under test.

When

- The field is submitted at exactly the documented minimum length, with all other fields valid.

Then

- The API returns HTTP `201 Created` if the minimum boundary is inclusive.

Note: This case remains pending until the actual field-length specification is approved.

TC-013 — Password and Confirm Password mismatch

Negative

Requirement: REQ-REG-06 Scenario: SC-13

Given

- The API contract confirms that Password and Confirm Password must match.
- All other fields are valid.

When

- The client sends different values for Password and Confirm Password.

Then

- The API returns HTTP `400 Bad Request`.

Note: The password-matching rule should be explicitly confirmed before this case is treated as an approved acceptance test.

4. Postman and Rest Assured execution approach

Test activity	Postman	Rest Assured
Positive registration	Send POST request and assert status/body	Send POST request and assert status/body

Mandatory field validation	Use data-driven requests with one missing field	Parameterize each mandatory field
Type validation	Modify JSON value types	Generate payload mutations
Boundary testing	Use collection variables and test data	Use parameterized BVA datasets
Negative validation	Assert HTTP 400	Assert HTTP 400 and approved error schema
Smoke testing	Tag and run the smoke collection	Run tagged smoke tests in the test suite
Traceability	Include requirement/test IDs in request names	Include IDs in feature/scenario names and reports

Recommended automation tags:

- `@HappyPath` — successful registration scenarios.
- `@Smoke` — minimum successful registration flow.
- `@Negative` — invalid payloads and rejected requests.
- `@Boundary` — minimum/maximum lengths and numeric-string boundary values.
- `@OptionalFields` — optional field combinations.

5. Download the complete test suite

Retail Banking Registration API — BDD and RTM

Excel workbook · 26 test cases · 2 worksheets

Includes:

- BDD test cases in Given–When–Then format
- Requirement and scenario traceability

- Positive, negative and boundary test coverage
- Happy Path and Smoke Testing tags
- Postman and Rest Assured execution guidance
- Requirement gaps and readiness status

Download the API BDD Test Suite and RTM

6. Final QA assessment

The test design covers the registration requirements supplied. However, full execution-ready coverage requires an approved API contract specifying the endpoint, exact JSON keys, per-field length limits, accepted formats, password policy, duplicate-registration rules and error response schema.

One important clarification: HTTP `500` conventionally means Internal Server Error, not specifically a timeout. The timeout response behavior should be explicitly agreed before finalizing TC-025.

The test cases are suitable for translating into Postman requests and Rest Assured BDD scenarios once these contract details are finalized.

Scenario 3: Test OTP login on mobile.

Persona]: As a senior software Automation Engineer, specialized in Appium and iOS automation and Principal Business Analyst [Instruction]: Write Testcases for Mobile App-OTP Login [Context]: [STRICT] Testcases should include both Functional, Non-Functional, OTP Expiry Time[5 min], OTP Format- 6 digits, Resend functionality available after 5 mins, network failure scenario, Appium/device context. [MANDTORY]: Should include all positive, negative, edge cases, timeout cases. RTM Should be there. Requirements-->Scenario-->Test Cases--->Test Steps--->Expected Results, should be clearly depicted with naming conventions as per industry standards. [DONOT]: Write irrelevant Test cases beyond the context or instruction. [Output]: Should be in clear, with industry standard and downloadable excel file [Tone]: Technical

Coverage included

Test area	Coverage

OTP login	Successful and unsuccessful verification
OTP format	Six digits, invalid length, non-numeric input, empty input and leading zero
OTP expiry	Validity before expiry and rejection at/after five minutes
Resend OTP	Before, at and after the five-minute threshold
Network failures	OTP request failure, verification failure and network recovery
Timeout scenarios	Delayed service responses and loading-state recovery
iOS and Appium	Device/simulator interactions, app background/foreground and UI state
Non-functional	Usability and reliability in the specified OTP flows
RTM	Requirement IDs mapped to scenarios and test cases

Automation and execution notes

- Appium: Automate OTP entry, verification, resend availability, screen transitions, and network-recovery behavior on supported iOS devices or simulators.
- Test data: Use an approved test OTP mechanism and a controllable clock or test service to validate five-minute boundaries reliably.
- RTM: Each test case includes a requirement ID, scenario ID, test case ID, BDD steps, expected results, automation notes, and clarification status.
- Release readiness: Confirm the precise expiry boundary, resend timer behavior, supported iOS/device matrix, and timeout thresholds before final acceptance.

The workbook is structured for functional and non-functional QA execution, with Happy Path, Smoke, Negative, Boundary, Network Failure and Timeout tags.